

Lab Practicum 01: Empirical Parallel Computing

Name Surname: Nuriya Sultanseitova

Student ID: 230103360

HOST CPU/SOC MODEL: Apple M4

OS & KERNEL: macOS (Darwin ARM64)

```
nuriya — -zsh — 76x24

Last login: Thu Sep 10 12:22:40 on ttys000
[nuriya@MacBook-Air-Nuria ~ % sysctl -a | grep machdep.cpu && sysctl hw.physi]
calcpu hw.logicalcpu hw.l1icachesize hw.l1dcachesize hw.l2cachesize
machdep.cpu.cores_per_package: 10
machdep.cpu.core_count: 10
machdep.cpu.logical_per_package: 10
machdep.cpu.thread_count: 10
machdep.cpu.brand_string: Apple M4
hw.physicalcpu: 10
hw.logicalcpu: 10
hw.l1icachesize: 131072
hw.l1dcachesize: 65536
hw.l2cachesize: 4194304
nuriya@MacBook-Air-Nuria ~ %
```

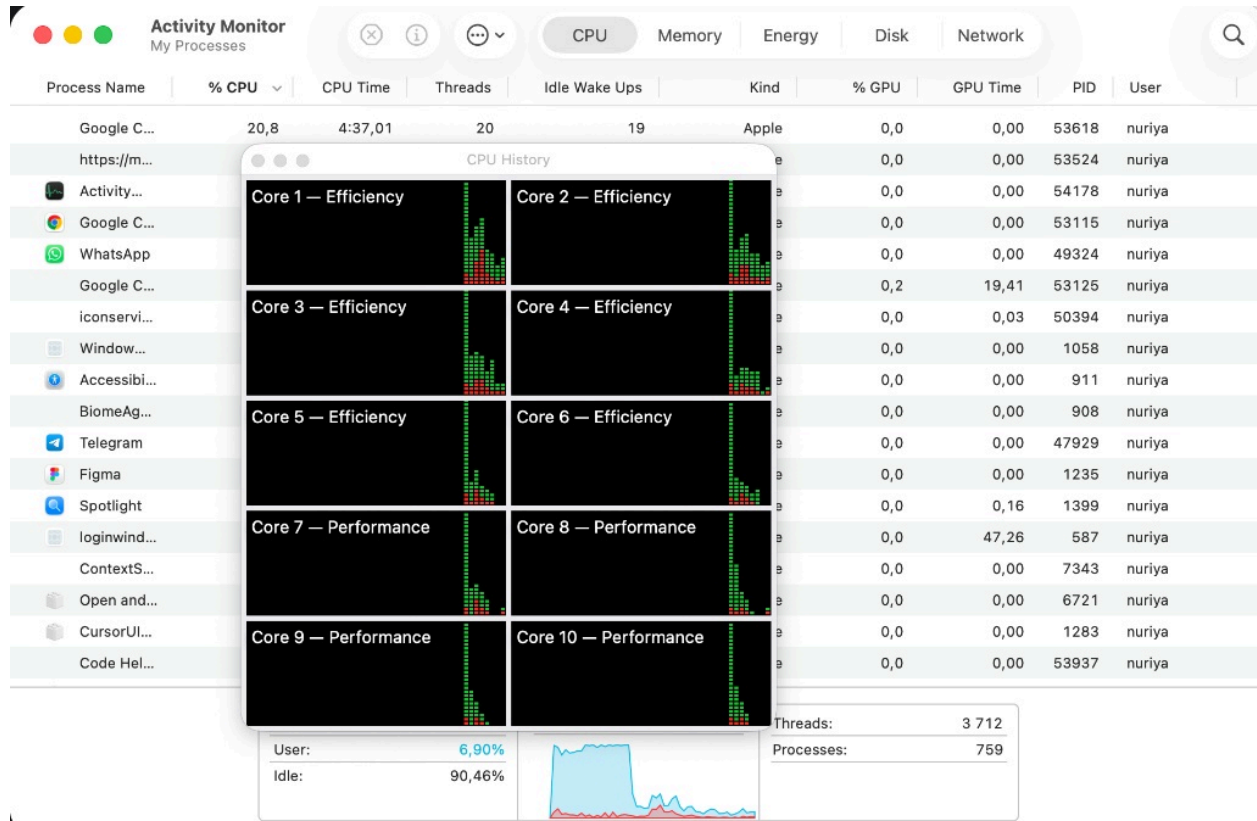
TASK 1:

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Apple M4 (ARM64)	Base & Max Boost Clock	Dynamic (~4.4 GHz Max)

Physical Hardware Cores	10	Logical Cores (SMT/Threads)	10
L1 Data/Instruction Cache	L1i: 128 KB / L1d: 64 KB	L2 Cache (Per Core / Total)	4 MB Total
L3 Shared Last-Level Cache	N/A (System Level Cache)	SIMD Vector Extensions	ARM NEON

Q1.1 Answer:

- **SISD (Single Instruction, Single Data):** Executing sequential, single-threaded instructions (e.g., standard single-threaded Python logic or compiler tasks) on a single physical core operating over a single data stream.
- **SIMD (Single Instruction, Multiple Data):** Executing parallel vector operations via ARM NEON vector instructions, processing multiple data elements simultaneously across vector registers within a single instruction cycle.
- **MIMD (Multiple Instruction, Multiple Data):** Executing multiple independent threads concurrently scheduled across the 10 physical cores of the Apple M4 processor, running distinct instruction streams over separate datasets.



TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

Language & Runtime: Python 3 (`multiprocessing.ProcessPoolExecutor`)

Workload Algorithm: Matrix Multiplication (200 \times 200)

Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN=T1/TN	Efficiency EN=SN/N
N=1(Baseline)	9.70	9.63	9.58	9.64	1.00x	100.0%
N=2	4.92	4.96	4.99	4.96	1.94x	97.2%
N=4	2.73	2.91	3.53	3.05	3.16x	79.0%

N=8	2.76	2.51	2.48	2.58	3.74x	46.7%
N=16	2.32	2.12	2.16	2.20	4.38x	27.4%
N=32	2.33	2.32	2.35	2.33	4.14x	12.9%

Q2.1: Scaling Saturation & Hardware Contention Analysis Speedup peaked at N=16 (4.38\times) and degraded at N=32 (4.14\times). The Apple M4 SoC consists of 10 physical cores (4 Performance cores + 6 Efficiency cores) without simultaneous multithreading (SMT). Scaling efficiency decreases past 4 workers as tasks spill onto lower-clocked Efficiency cores. When $N > 10$, workers exceed physical core capacity, introducing OS context-switching overhead, cache thrashing across shared L2 memory, and CPU thermal throttling under sustained multi-core load.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

Run	Measured Output	Error (107-Act)	Run	Measured Output	Error (107-Act)
#1	10000000	0	#6	10000000	0
#2	10000000	0	#7	10000000	0
#3	10000000	0	#8	10000000	0
#4	10000000	0	#9	10000000	0
#5	10000000	0	#10	10000000	0

Q3.1: Read-Modify-Write (RMW) Window At the hardware level, incrementing a shared memory variable requires three non-atomic steps: **Load** (read value from RAM into core register), **Add** (increment value), and **Store** (write back to memory). When multi-core threads execute simultaneously, thread interleaving within this RMW window causes stale reads and overwritten updates. In standard CPython,

thread execution is serialized by the Global Interpreter Lock (GIL), which prevents true multi-core parallel thread execution for native bytecode, resulting in zero measured error count in Python threads.

Q3.2: The Synchronization Penalty Unlocked = 166.45 ms vs. Locked = 562.89 ms

Synchronized code is significantly slower (~3.4x execution penalty) because acquiring and releasing mutexes forces thread serialization (queuing threads sequentially), induces locking overhead, causes L1/L2 cache-line invalidation (cache bouncing), and triggers OS kernel-level thread context switches.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

1. Parallel Fraction Derivation (p)	2. Amdahl Theoretical Speedup Ceiling
Formula: $p = 2 * (T1 - T2) / T1$ $T1 = 9.64 \text{ s}, T2 = 4.96 \text{ s}$ Calculated p = 0.9710	Formula: $S_{\text{max}} = 1 / (1 - p)$ Sequential fraction $(1 - p) = 0.0290$ Max Theoretical Speedup (N → infity) = 34.43x

3. Amdahl 64-Core Projection	4. Gustafson Scaled Speedup (Weak Scaling)
Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$ Projected Speedup on 64 Cores = 22.62x	Formula: $S(\text{Gustafson}) = (1 - p) + (p * 64)$ Projected Scaled Speedup (64x Problem) = 62.17x

Q4.1: Strong vs. Weak Scaling Analysis

Amdahl's model assumes **Strong Scaling** (fixed problem size), where the unparallelizable sequential fraction ($1-p = 0.0290$) creates a hard theoretical upper bound ($S_{\max} = 34.43$) regardless of core count. Gustafson's model assumes **Weak Scaling** (problem size expands proportionally with processing cores). Because parallel workload scales up while absolute serial overhead remains constant, Gustafson predicts near-linear speedup (62.17 on 64 cores).